

Security Scan Report

Target: /fake/target

Scan Date: August 26, 2026 at 19:32 UTC

Scan ID: f9c222c3-f012-4db6-b36f-0acd3a5aa4a8

0 CRITICAL

1 HIGH

0 MEDIUM

2 LOW

Executive Summary

Security scan of '/fake/target' identified 4 raw findings, reduced to 3 unique issues after deduplication. ■ 1 HIGH-severity issue(s) should be prioritized for remediation within the next 48 hours. 2 LOW-priority issue(s) have been logged for backlog tracking.

Scan Statistics

Metric	Value
Total Raw Findings	4
After Deduplication	3
Critical Findings	0
High Findings	1
Medium Findings	0
Low Findings	2
Scan Duration	0.0s

Detailed Findings

[HIGH] #1: Sql Injection in users.py:42

Risk Score	0.5488
Affected Location	app/routes/users.py:42
Vulnerability Type	SQL_INJECTION
CWE ID	CWE-89
Detected By	semgrep

Explanation
SQL Injection occurs when user-controlled input is directly interpolated into SQL queries, allowing an attacker to alter query logic. Found in users.py.

Code or HTTP Excerpt

```
42 | query = f"SELECT * FROM users WHERE id = {user_id}"
```

Business Impact
An attacker could read, modify, or delete user accounts and personal data, potentially exfiltrating the entire database including credentials and PII.

Fix Recommendation
Best Fix: Use parameterized/prepared queries instead of string concatenation. Pass user input as bind parameters, not as part of the query string.

[LOW] #2: Xss in render.py:15

Risk Score	0.3478
Affected Location	app/utils/render.py:15
Vulnerability Type	XSS
CWE ID	CWE-79
Detected By	semgrep

Explanation
Cross-Site Scripting (XSS) allows attackers to inject malicious scripts into web pages viewed by other users. Found in render.py.

Business Impact
An attacker could steal session tokens from user accounts and personal data, redirect users to phishing pages, or perform actions as the victim.

Fix Recommendation
Best Fix: Apply context-aware output encoding before rendering user content. Use your framework's built-in auto-escaping.

[LOW] #3: Hardcoded Secret in settings.py:8

Risk Score	0.3168
Affected Location	config/settings.py:8
Vulnerability Type	HARDCODED_SECRET
Detected By	semgrep

Explanation

Hardcoded credentials, API keys, or tokens in source code can be extracted by anyone with repository access. Found in settings.py.

Business Impact

The exposed credentials could grant unauthorized access to user credentials and passwords, enabling data theft or service abuse.

Fix Recommendation

Best Fix: Move the password to an environment variable. Use `os.environ.get('DB_PASSWORD')` instead of the literal string. Rotate the exposed password immediately.

Appendix

Deduplication Statistics

Total raw findings: 4 | After deduplication: 3 | Reduction: 25%

Scan Metadata

Scan ID	f9c222c3-f012-4db6-b36f-0acd3a5aa4a8
Target	/fake/target
Timestamp	2026-08-26T19:32:04.021220+00:00
Duration	0.0s
Scanners Used	unknown
Scanner Versions	unknown

Raw Findings Data (JSON)

```
[
  {
    "id": "2a5ffd4f-6382-4305-8334-cc02a8e524d0",
    "title": "Sql Injection in users.py:42",
    "file": "app/routes/users.py",
    "line_start": 42,
    "line_end": null,
    "severity": "HIGH",
    "vuln_type": "SQL_INJECTION",
    "description": "SQL injection via string concatenation",
    "rule_id": "python.lang.security.sqli",
    "confidence": 0.9,
    "cwe_id": "CWE-89",
    "cvss_score": null,
    "code_snippet": "query = f\"SELECT * FROM users WHERE id = {user_id}\"",
    "sources": [
      "semgrep"
    ],
    "risk_score": 0.5488,
    "risk_category": "HIGH",
    "category_source": "vuln_floor",
    "category_reason": "minimum floor for SQL_INJECTION",
    "llm_explanation": "SQL Injection occurs when user-controlled input is directly interpolated into SQL queries",
    "fix_suggestion": "Use parameterized/prepared queries instead of string concatenation. Pass user input as a parameter.",
    "business_impact": "An attacker could read, modify, or delete user accounts and personal data, potentially leading to data breaches or account hijacking.",
    "attack_path": "UNKNOWN: deterministic fallback has no runtime proof.",
    "next_test": "Run one authorized, read-only validation test and stop if it changes state or exposes read-only data.",
    "raw": {}
  },
  {
    "id": "23be324d-c86a-4b29-b8d5-962ab9f5eaa6",
    "title": "Xss in render.py:15",
    "file": "app/utils/render.py",
    "line_start": 15,
    "line_end": null,
    "severity": "MEDIUM",
    "vuln_type": "XSS",
    "description": "Unescaped user input in template",
    "rule_id": "python.flask.security.xss",
    "confidence": 0.8,
    "cwe_id": "CWE-79",
    "cvss_score": 5.4,
    "code_snippet": "return render_template('index.html', user_input=user_input)",
    "sources": [
      "semgrep"
    ],
    "risk_score": 0.32,
    "risk_category": "MEDIUM",
    "category_source": "vuln_floor",
    "category_reason": "minimum floor for XSS",
    "llm_explanation": "XSS occurs when user input is rendered in the browser without proper escaping.",
    "fix_suggestion": "Escape user input using Jinja2's autoescaping feature or manually escape HTML entities.",
    "business_impact": "An attacker could execute malicious scripts in the browser, leading to session hijacking or defacement.",
    "attack_path": "UNKNOWN: deterministic fallback has no runtime proof.",
    "next_test": "Test for XSS by injecting a script tag into the user input field.",
    "raw": {}
  }
]
```

```

    "confidence": 0.7,
    "cwe_id": "CWE-79",
    "cvss_score": null,
    "code_snippet": null,
    "sources": [
        "semgrep"
    ],
    "risk_score": 0.3478,
    "risk_category": "LOW",
    "category_source": "score",
    "category_reason": "score-threshold",
    "llm_explanation": "Cross-Site Scripting (XSS) allows attackers to inject malicious scripts into web pages.",
    "fix_suggestion": "Apply context-aware output encoding before rendering user content. Use your framework's built-in escaping functions.",
    "business_impact": "An attacker could steal session tokens from user accounts and personal data, redirecting them to malicious sites.",
    "attack_path": "UNKNOWN: deterministic fallback has no runtime proof.",
    "next_test": "Run one authorized, read-only validation test and stop if it changes state or exposes read-only data.",
    "raw": {}
},
{
    "id": "1ae13130-f0aa-4bb6-9f9b-1a2e0ff78fac",
    "title": "Hardcoded Secret in settings.py:8",
    "file": "config/settings.py",
    "line_start": 8,
    "line_end": null,
    "severity": "LOW",
    "vuln_type": "HARDCODED_SECRET",
    "description": "Hardcoded password detected",
    "rule_id": "python.lang.security.hardcoded-password",
    "confidence": 0.4,
    "cwe_id": null,
    "cvss_score": null,
    "code_snippet": null,
    "sources": [
        "semgrep"
    ],
    "risk_score": 0.3168,
    "risk_category": "LOW",
    "category_source": "score",
    "category_reason": "score-threshold",
    "llm_explanation": "Hardcoded credentials, API keys, or tokens in source code can be extracted by anyone with access to the code.",
    "fix_suggestion": "Move the password to an environment variable. Use os.environ.get('DB_PASSWORD') instead of a hardcoded string.",
    "business_impact": "The exposed credentials could grant unauthorized access to user credentials and passwords.",
    "attack_path": "UNKNOWN: deterministic fallback has no runtime proof.",
    "next_test": "Run one authorized, read-only validation test and stop if it changes state or exposes read-only data.",
    "raw": {}
}
]

```